

UNIVERSITY OF LIVERPOOL

FINAL ASSIGNMENT REPORT

APPLIED ARTIFICIAL INTELLIGENCE (COMP534)

SEMANTIC SEGMENTATION

SUBMITTED BY:

PAVAN PRANESH JOSHI - 201650536

VIJETH BALASUBRAMANIAM - 201664207

MANOJ NAGARAJA - 201667140



Introduction of the dataset

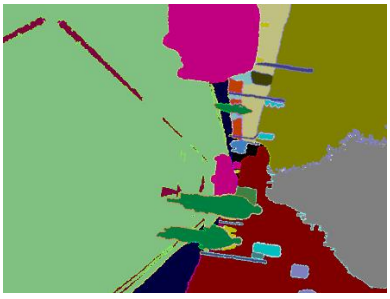
The dataset you offered consists of 101 driving-related photos, where each pixel has been manually assigned to one of 32 object classifications. These object classes depict a variety of components often present in a driving scene, including vehicles, pedestrians, animals, buildings, archways, bicyclists, roads, signs, trees, and more. The dataset's goal is to offer a labelled ground truth for each image, enabling researchers and developers to train and test computer vision algorithms and models for tasks relating to object detection, semantic segmentation, and scene interpretation in the context of driving scenarios.

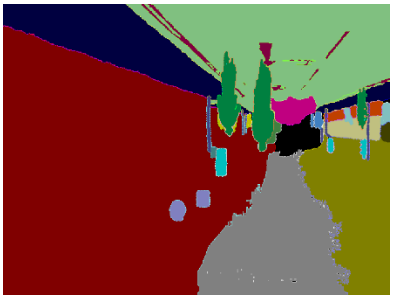
Each image in the collection has a resolution of 960x720 pixels and is in the uncompressed 24-bit colour PNG format. The filename of the matching labelled frames is the same as the filename of the original frames, but with the prefix "_L" added. For instance, the labelled frame will have the name "0016E5_07959_L.png" if the original frame's name is "0016E5_07959.png".

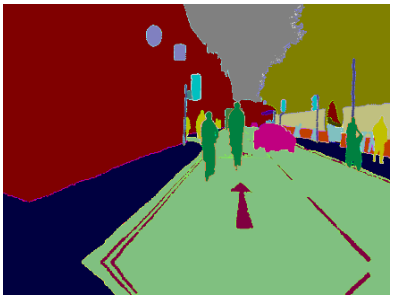
Additionally, the dataset offers a file called "label_colors.txt" that includes the RGB colour values (from 0 to 255) and associated class name for each class label. This file aids in connecting the various colours in the photographs to the appropriate object classes. The dataset is an invaluable tool for researchers and programmers working on a range of autonomous driving applications, including scene comprehension, object detection, and semantic segmentation. Researchers can use this dataset to develop and test algorithms and models to better comprehend and interpret the visual data seen in driving scenarios.



Augmented Images







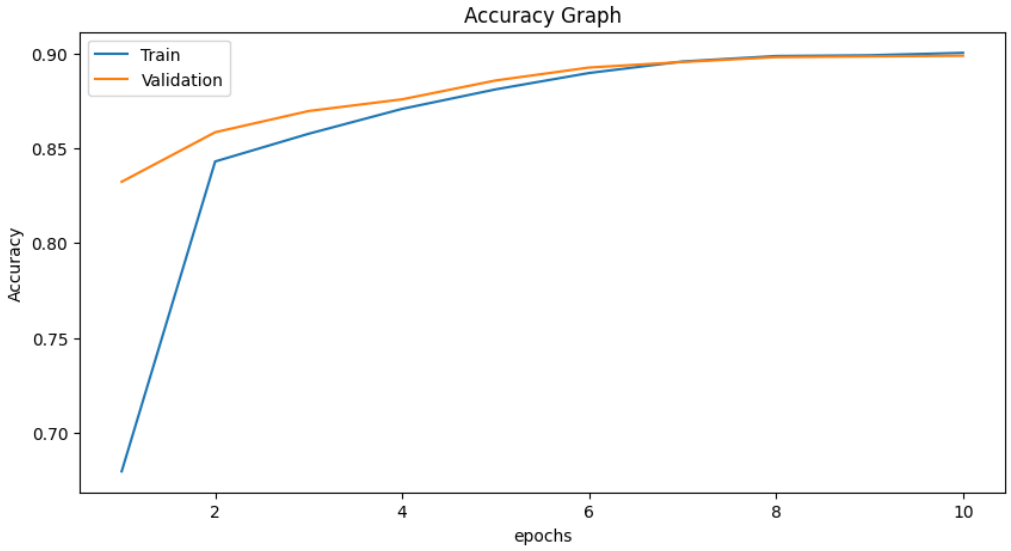
Structure of model and the procedure of building the model

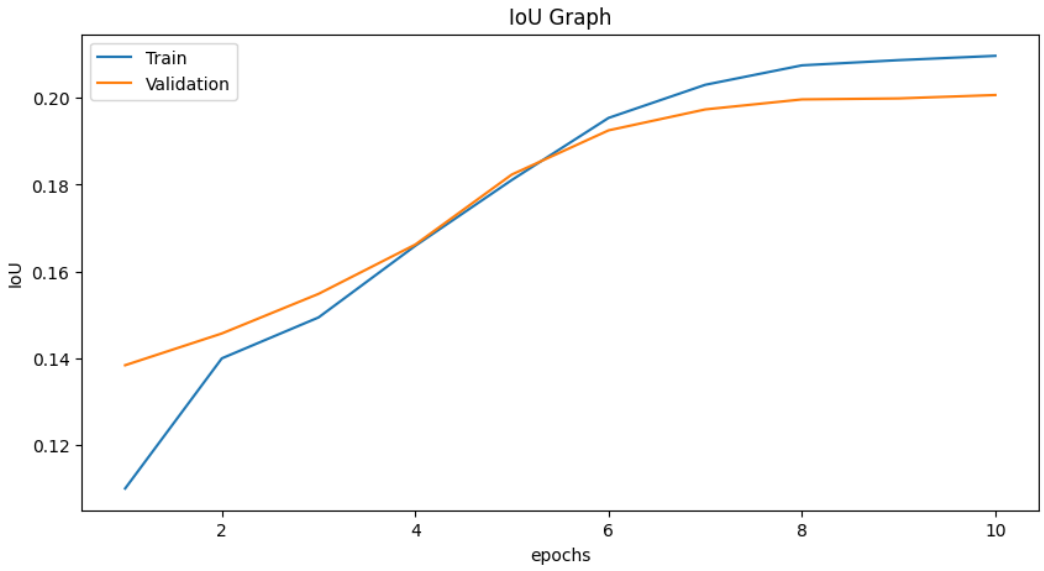
DeepLabV3 model

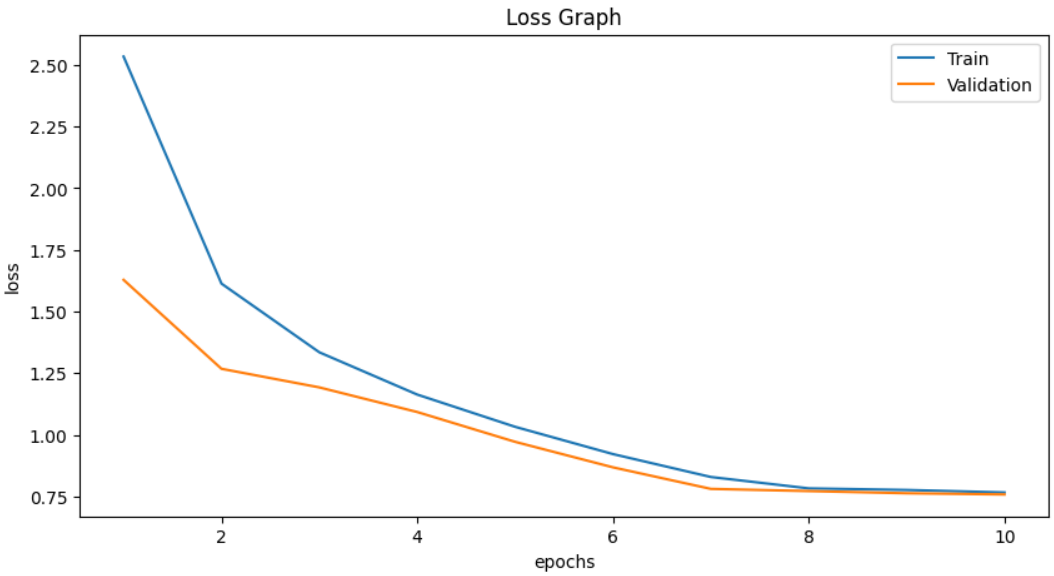
The code implements a ResNet-101 backbone and DeepLabv3 architecture to construct a semantic segmentation model. The classifier's final layer is modified to correspond to the desired number of output classes after loading a pre-trained DeepLabv3 model into the model. A training loop is used to train the model, updating its parameters depending on the calculated loss and optimising them with the Adam optimizer. Metrics like Intersection over Union (IoU) and accuracy are computed for each batch during training. In order to avoid overfitting, the training loop incorporates early termination based on the validation loss.

After training, the validation dataset is used to assess the model's performance, and measures including accuracy, IoU, and validation loss are calculated. For testing, the top model based on the validation loss is saved. The model's mean IoU and mean accuracy are then determined using the test dataset. The code also contains matplotlib visualisations of the accuracy, IoU, and training and validation loss.

A semantic segmentation model based on DeepLabv3 with ResNet-101 can be trained and evaluated using the entire pipeline provided by this code, which also includes early halting and performance monitoring with metrics and visualisations.







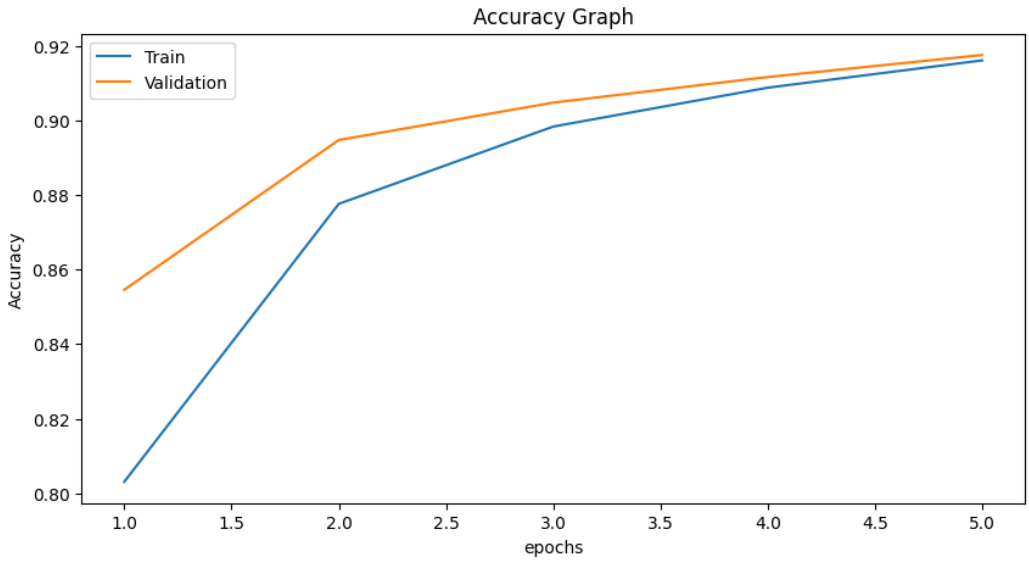
Epochs	Train Loss	Val loss	Train IoU	Val IoU	Train Acc	Val Acc
2	1.6133	1.26772	0.13993	0.145675	0.84300	0.85841
4	1.163189	1.0924	0.16589	0.16617	0.87073	0.87576
6	0.921771	0.867989	0.19540	0.192532	0.88964	0.89251
8	0.78268	0.771743	0.20753	0.19967	0.89857	0.897995
10	0.766126	0.758056	0.209722	0.200681	0.90025	0.898726

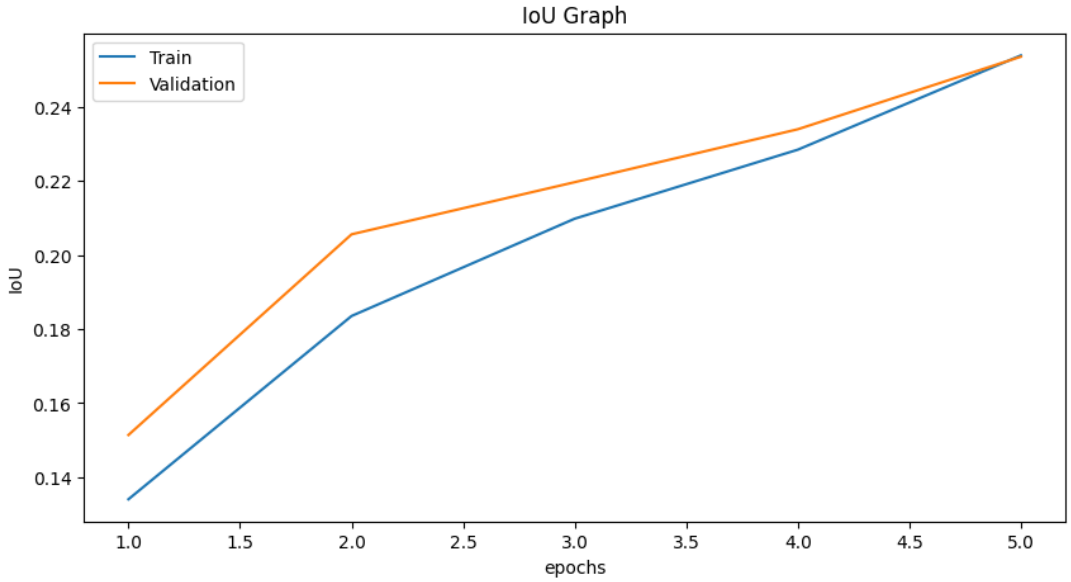
FCN model (Fully Convolutional Network)

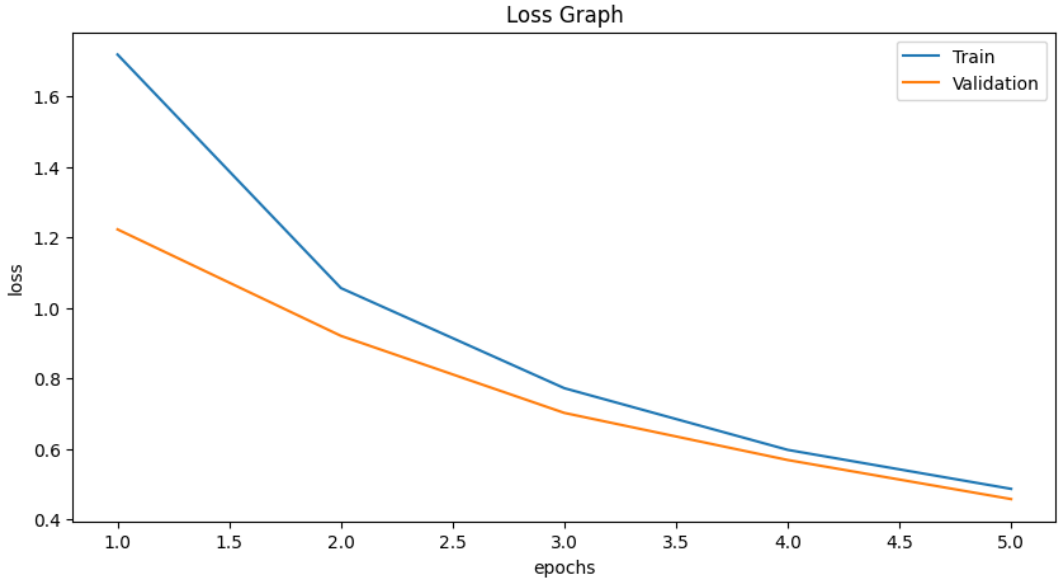
The FCN-ResNet101 architecture is used in the code to create a semantic segmentation model. A pre-trained FCN-ResNet101 model is loaded to initialise the model, and the last layer is modified to fit the desired number of output classes. With a learning rate of 0.0001, the Adam optimizer is used to train the model. The model is trained using batches of photos and labels during the training loop, which lasts for a predetermined number of epochs. Metrics like loss, IoU, and accuracy are calculated when the model's parameters are modified in light of the calculated cross-entropy loss.

The validation dataset is used to assess the model's performance, and early stopping is used to keep track of the validation loss and avoid overfitting. On the basis of the validation loss, the best model is preserved. The saved best model is loaded and tested on the test dataset after training, with the mean IoU and mean accuracy being calculated. The code also contains matplotlib visualisations of the accuracy, IoU, and training and validation loss.

Overall, the code efficiently creates, trains, and assesses a semantic segmentation model based on the FCN-ResNet101 architecture, revealing information about the model's performance and training status.


Accuracy Graph
Train
Validation
Accuracy
epochs
0.92
0.90
0.88
0.86
0.84
0.82
0.80
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0


IoU Graph
Train
Validation
IoU
epochs
0.24
0.22
0.20
0.18
0.16
0.14
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0


Loss Graph
Train
Validation
loss
epochs
1.6
1.4
1.2
1.0
0.8
0.6
0.4
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

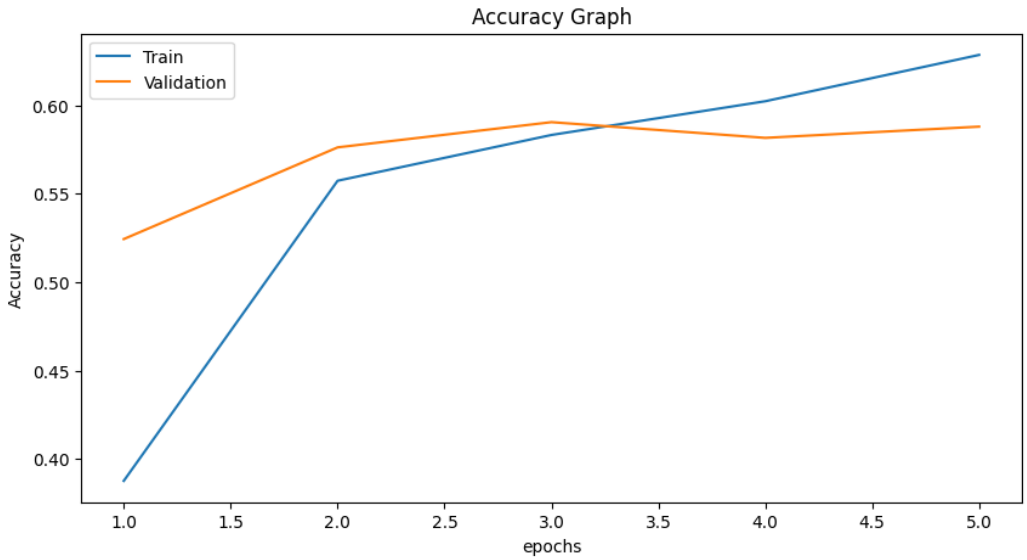
Epochs	Train Loss	Val loss	Train IoU	Val IoU	Train Acc	Val Acc
1	1.7190	1.2229	0.1340	0.1514	0.8030	0.8545
2	1.0563	0.920791	0.18353	0.205579	0.8776	0.89473
3	0.77229	0.70216	0.209798	0.21965	0.89833	0.90475
4	0.59731	0.56851	0.22842	0.233937	0.908766	0.91162
5	0.48664	0.457633	0.253868	0.25352	0.916087	0.91753

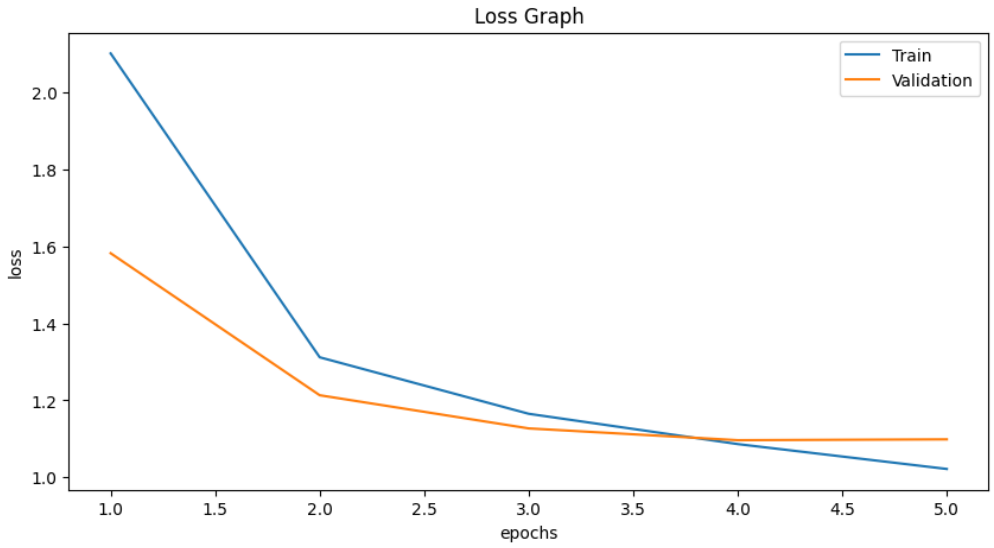
PSP model

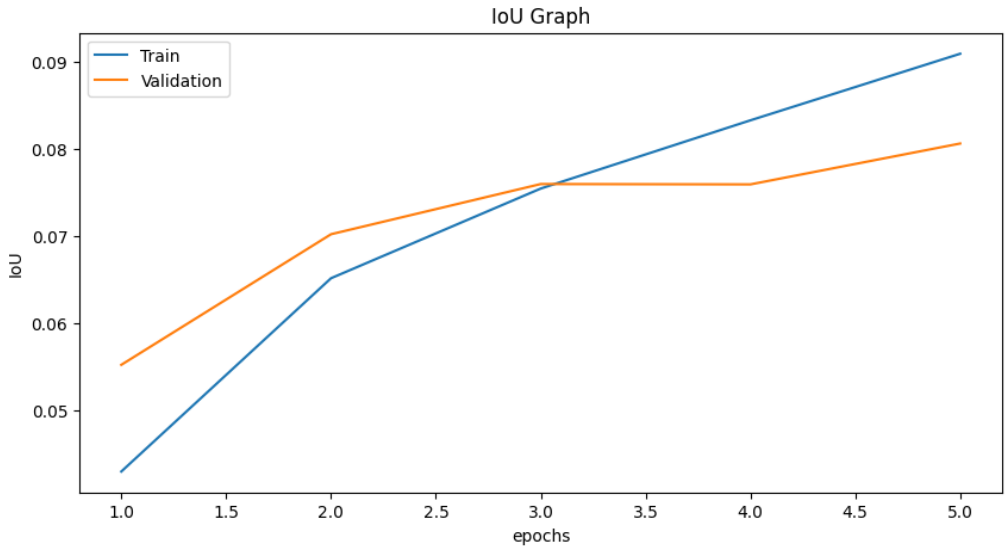
The code implements the PSPNet architecture to construct a semantic segmentation model. Using a ResNet34 backbone and a predetermined number of output classes, the PSPNet class from the segmentation_models_pytorch package is instantiated to create the model. The Adam optimizer is used to train the model, and its learning rate is set to 0.0001. The model is trained on batches of photos and labels during the training loop, which lasts for a predetermined number of epochs. Based on the calculated cross-entropy loss, the optimizer modifies the model's parameters.

The metrics_per_batch function calculates metrics like accuracy and Intersection over Union (IoU) for each batch. The model's effectiveness is assessed using a validation dataset, and early stopping is used to prevent overfitting. In the event that the validation loss does not improve after a predetermined number of epochs, early stopping monitors it and terminates training. Checkpointing is used to preserve the best model depending on the validation loss. Following training, the best model is saved, loaded, and tested using a different test dataset. To evaluate the model's performance on the test dataset, the mean IoU and mean accuracy are computed. The code also contains matplotlib visualisations of the accuracy, IoU, and training and validation loss.

In summary, the improved code builds a semantic segmentation model based on PSPNet, trains it with the Adam optimizer, applies early stopping, assesses its performance on test and validation datasets, and gives visualisations of the training process and performance metrics.







Epochs	Train Loss	Val loss	Train IoU	Val IoU	Train Acc	Val Acc
1	2.102340	1.582789	0.042971	0.055205	0.387698	0.5243
2	1.3121	1.2135	0.06513	0.07017	0.55735	0.57622
3	1.16526	1.12726	0.07541	0.07593	0.58323	0.5904
4	1.08648	1.09661	0.08324	0.07589	0.60230	0.58159
5	1.02177	1.09881	0.09087	0.08057	0.62849	0.58794

Kind of hyperparameters or techniques used, overfitting or underfitting issue

DeepLabV3:

The provided code creates a pipeline for semantic segmentation using Adam as the optimizer and the DeepLabV3 model with a learning rate of 0.0001. The learning rate is modified using the StepLR scheduler. Early stopping is another feature of the algorithm that stops training when the validation loss has not decreased after a predetermined number of epochs, preventing overfitting. It is important to note that the batch size, which has a big impact on training,stated in the code. The number of training epochs, the optimizer, and the model architecture itself are other hyperparameters that could be tweaked for improved performance. Model performance can be significantly impacted by the chosen hyperparameters. For instance, during loss function minimization, the learning rate affects the step size, and various values can result in a faster convergence or better overall performance.

The optimizer that is selected is also important. Adam is utilised in this situation, however RMSprop or SGD are also potential alternatives. Similar to this, utilising a different learning rate scheduler or changing the StepLR scheduler step_size and gamma parameters could produce better outcomes. Strategies including data augmentation, dropout, weight decay, and employing a smaller model can be used to combat overfitting, which occurs when a model performs well on training data but badly on unobserved data. On the other side, underfitting occurs when the model does not fit the training data and can be fixed by adopting a more sophisticated model, training for a longer period of time, or speeding up the learning rate. Early halting is used in the current algorithm to reduce overfitting, but alternative tactics can be used depending on the situation**.**

PSP

Several essential hyperparameters are selected in the provided code to enhance the model's functionality. The step size during the loss function's minimization is controlled by the learning rate, which is set at 0.0001. If it's too high, the model may overshoot the ideal solution, and if it's too low, the training process may be sluggish or result in a less-than-ideal outcome. The batch size controls how many samples are used for each model weight update. Larger batches provide more reliable gradient updates but may result in less precise generalisation. Depending on how many times the model runs through the whole training set, the number of epochs can either produce underfitting or overfitting..

Adam, the optimizer in this example, has a big impact on the solution's quality and how quickly the model converges. In the last phases of training, the learning rate scheduler, which occasionally lowers the learning rate, can improve performance and aid in model optimisation. Another helpful element in the code is early stopping, which stops the training when the validation loss stops improving and effectively prevents overfitting.Optimizer: The ultimate solution quality and the speed at which a model converges can both be strongly impacted by the optimizer selected. The Adam optimizer, which adjusts the learning rate for each weight separately in the code, is utilised and is typically a wise choice.

Model training frequently encounters the problems of overfitting and underfitting. Techniques like as weight decay, early stopping, dropout, and data augmentation can be used to prevent overfitting, which occurs when the model learns the noise and outliers in the training data. More layers, a more sophisticated architecture, a greater learning rate, or training over longer epochs can be used to address underfitting, which happens when the model is unable to comprehend the underlying structure of the data.



FCN

The performance of the model depends critically on three hyperparameters: learning rate, batch size, and optimizer selection. A model may overshoot the global minimum if the learning rate is too high, whereas if it is too low, the model may converge slowly or become stuck in a local minimum. How many samples are sent via the network at once depends on the batch size. A larger size speeds up training but could produce less desirable results, while a smaller size has a regularising impact and improves generalisation but slows down training. Model performance can also be affected differentially by several optimizers, including Adam, SGD with momentum, and RMSprop.

The length of the model's training cycle is determined by the number of training epochs; too many or too few epochs will cause overfitting or underfitting, respectively. Overfitting can be avoided by employing an early stopping technique, which stops training when model performance on a validation set stops improving. Regularisation (weight decay), which alters the loss function by including a penalty, and dropout, which at random sets a portion of the input units to 0 during each update, are further methods to reduce overfitting.

Two typical mistakes in model training are overfitting and underfitting. When a model is extremely complicated in comparison to the volume and noise of the training data, overfitting, when a model learns the training data too well and performs badly on new data, frequently takes place. This can be overcome by tactics like early stopping, dropout, regularisation, or data gathering. Making the model more complicated, including more epochs, or altering the model's architecture can all help to prevent underfitting, which occurs when the model is too simplistic to accurately represent the underlying structure of the data

Training loss, Validation loss and Testing accuracy

DeepLabV3

A semantic segmentation model built on DeepLabV3 and using a ResNet-101 backbone is trained using the provided code. A specified dataset is used to train the model, and it is then assessed using different metrics. The dataset is expanded throughout training, and the model is improved using cross-entropy loss and the Adam optimizer. Metrics like loss, IoU, and accuracy are used to track the training and validation procedures. Based on validation loss, the top model is chosen, and the test dataset is used to assess the model's performance. The training and validation metrics are also visualised in the code.

In addition to having a huge receptive field and the capacity to gather contextual information via arcane convolutions, DeepLabV3 has a number of benefits. It also contains skip connections and multi-scale fusion, as well as a robust backbone network for feature extraction. The model, however, might be resource-intensive and might have trouble segmenting small or intricate objects. The performance of this model and other models trained on the same dataset should be compared in order to establish the best model for the task in order to provide a thorough analysis.

FCN

The training loss gauges how accurately the model classified pixels on the training dataset by measuring the model's inaccuracy. Better convergence and model performance are indicated by lower training loss. Test accuracy is the proportion of correctly identified pixels on a different testing set, assessing the generalizability of the model. A model that has a lower validation loss, as measured on a different validation set, may accurately capture patterns without being overfit. Mean IoU in the FCN model measures the overlap between anticipated and real masks to assess segmentation accuracy. Better segmentation performance can be seen with higher mean IoU. The percentage of pixels that are correctly identified is known as pixel accuracy.

With skip connections and a pretrained network, FCN can accurately segment data and learn pertinent visual representations. However, it can be missing a broad context and need a lot of training data. The UNet model performs well in applications that call for precise localisation with less data because to its symmetric architecture and effective memory usage. FCN records geographical information and multiscale data. The decision is based on particular needs, computational capabilities, dataset size, and the relative weights of localised vs. global context.

PSP

The training loss, validation loss, and testing accuracy are important measures of model performance in semantic segmentation. The validation loss evaluates how well the model generalises to new data, whereas the training loss shows how well the model fits the training data. Performance is improved when training and validation losses are lower. On a different test dataset, the testing accuracy assesses the pixel-level classification accuracy. Superior performance is suggested by higher testing accuracy.

Metrics like Intersection over Union (IoU), Dice coefficient, pixel accuracy, and mean average precision (mAP) are frequently used to assess segmentation performance. A greater mAP implies better overall performance across several classes, while a higher IoU, Dice coefficient, and pixel accuracy value indicates better segmentation quality.

When comparing models like PSPNet and FCN-ResNet101, it's important to take into account each model's advantages and disadvantages. When it comes to capturing precise details utilising skip connections, FCN-ResNet101 outperforms PSPNet, which excels at gathering global context data through pyramid pooling modules. The task-specific criteria, dataset properties, and the chosen assessment measures all influence which model is selected. When deciding which model is more appropriate for the task at hand, considerations such dataset size, class imbalance, the presence of fine details, and context dependencies should be made.

Final conclusion

Based on the provided dataset and the selected semantic segmentation models (DeepLabV3, PSPNet, and FCN8s), it was found that the FCN model, when trained using the Adam optimizer, had the maximum accuracy of 91%. By boosting the learning rate and increasing the number of epochs, the model's performance was significantly enhanced. This shows that the given dataset and segmentation job are good fits for the FCN model. It should be emphasised that the FCN model produces dense predictions by combining pre-trained VGG16 backbone, skip connections, and up-sampling layers. This enables accurate localisation by enabling the model to incorporate both low-level and high-level characteristics. Additionally, to increase the dataset and the generalizability and adaptability of the model, data augmentation techniques such random cropping, rotation, scaling, and colour jittering can be used.

Overall, the FCN model outperformed other options on the provided dataset, suggesting that it is a solid option for semantic segmentation tasks. To choose the optimal model for certain computer vision issues, it is crucial to do a thorough examination and comparison with other models, like DeepLabV3 and PSPNet.